

Operating System

Prepared By:

Er. Ayush Chaudhary

Chapter 5

File Management

Contents:

File Overview:	4
Files	4
File Naming:	5
File Structure:	6
File Type:	8
File Access:	12
File Attributes:	14
File Operations:	15
File System Layout:	16
File Directories:	17
Directory Operation:	18
Single-Level Directory:	20
Path Names:	22
Directory Implementation:	23
Shared File:	26
Implementing Files:	27
Contiguous Allocation:	27

Linked List Allocation:	27
Linked List Allocation Using a Table in Memory:	29
I-Node:.....	32

File Overview:

Files

- File: a named collection of data that may be manipulated as a unit by operations such as:
 - Open, Close, Create, Destroy, Copy, Rename, List
- Individual data items within a file may be manipulated by operations like:
 - Read
 - Write
 - Update
 - Insert
 - Delete
- File characteristics include:
 - Location
 - Accessibility
 - Type
 - Volatility

- Files can consist of one or more records

File Naming:

- Probably the most important characteristic of any abstraction mechanism is the way the objects being managed are named. When a process creates a file, it gives the file a name. When the process terminates, the file continues to exist and can be accessed by other processes using its name.
- The **exact rules for file naming vary somewhat from system to system**, but all current operating systems **allow strings** of one to eight letters as legal file names. Frequently digits and special characters are also permitted. *Many file systems support names as long as 255 characters.*

Some file systems distinguish between upper- and lower-case letters, whereas others do not. UNIX (including all its variants) falls in the first category; MS-DOS falls in the second

File Structure:

The file can be structured in many ways. The most common structures are:

1. Unstructured:

- In an unstructured file structure, **data is stored without any predefined format** or organization.
- This means **that there are no rules or constraints** dictating how data should be arranged within the file.
- Unstructured files **typically consist of a sequence of bytes or characters**, with no inherent hierarchy or relationships between the data elements.
- Examples of unstructured files include plain text files, binary files, and multimedia files like images or videos.

2. Record Structure:

- In a record structure, data is organized into records, where **each record represents a unit of information with a predefined format**.

- Records may consist of fields or attributes that store specific types of data, and there may be rules governing the arrangement and formatting of these fields within the record.
- Common examples of record-based file formats include CSV (Comma-Separated Values), where each line represents a record with fields separated by commas, and database files like SQL databases, where tables are composed of records with predefined schemas.

3. Tree Structure:

- A tree structure organizes data hierarchically, similar to the structure of a tree with branches and nodes.
- Each node in the tree represents a data element, and nodes may have parent-child relationships, forming a hierarchical structure.

In summary, unstructured files lack a predefined format, record structures organize data into predefined records, and tree structures organize data hierarchically in a tree-like fashion.

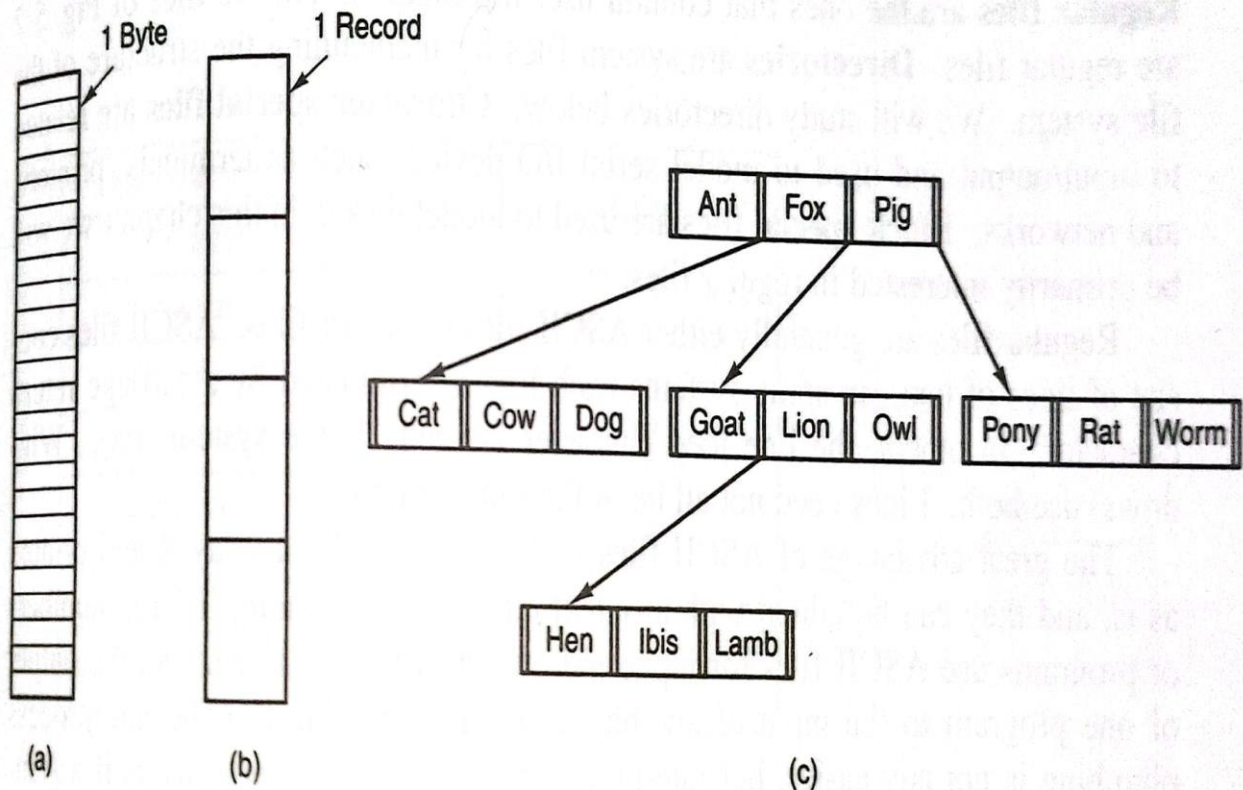


Figure: (a) Unstructured. (b) Record. (c) Tree

File Type:

File type refers to the ability of the operating system to distinguish different types of file such as text files source files and binary files etc.

Many operating systems support many types of files.

Operating system like MS-DOS and UNIX have the following types of files –

1.Ordinary/Regular files:

- These are the files that contain user information.
- These may have text, databases or executable program.
- The user can apply various operations on such files like add, modify, delete or even remove the entire file.
- **Example:** example.txt

This is a simple text file that contains user-generated content, such as notes, documents, or configuration settings.

2.Directory files:

- These files contain list of file names and other information related to these files.
- **Example:** /home/user/Documents

This is a directory that contains other files and directories.

3.Special files:

- These files are also known as device files.
- These files represent physical device like disks, terminals, printers, networks, tape drive etc.

These files are of two types –

- **Character special files** – data is handled character by character as in case of terminals or printers.

- **Example:** /dev/ttyUSB0 (on UNIX-like systems)

This file represents a USB serial port, allowing character-by-character communication with a connected device, such as a microcontroller or modem.

- **Block special files** – data is handled in blocks as in the case of disks and tapes.

- **Example:** /dev/sda (on UNIX-like systems)
- This file represents a physical disk drive, allowing direct access to the storage device.

file type	usual extension	function
executable	exe, com, bin or none	read to run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rrf, doc	various word-processor formats
library	lib, a, so, dll, mpeg, mov, rm	libraries of routines for programmers
print or view	arc, zip, tar	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm	binary file containing audio or A/V information

File Access:

- **Sequential Access:** The simplest access method is **sequential access**. Information in the file is processed in order, one record after the other. This mode of access is by far the most common; for example, editors and compilers usually access files in this fashion.

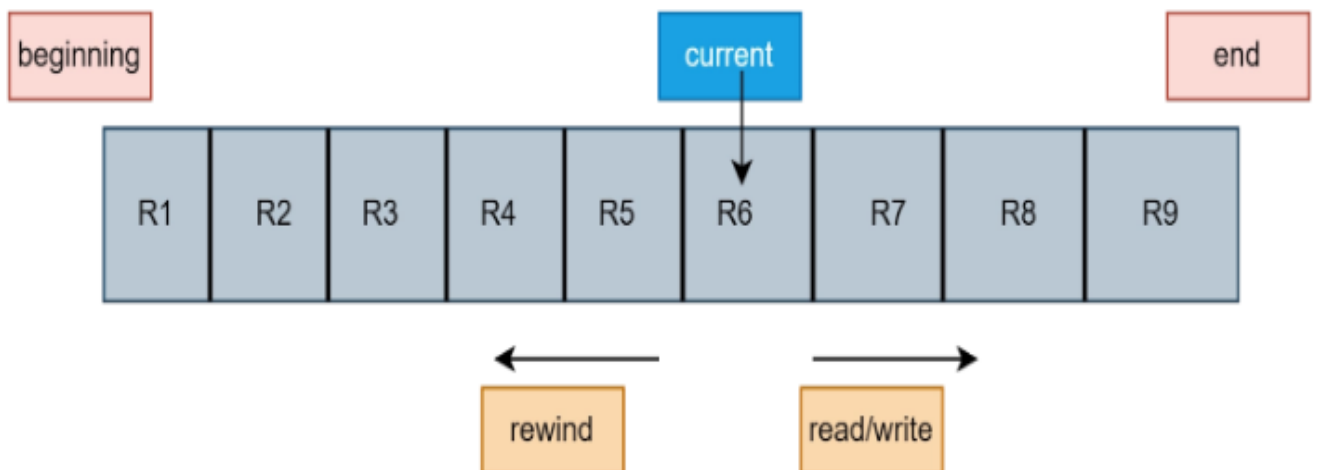
read next

write next

Reset

no read after last write

(rewrite)



- **Direct/Random Access:** Another method is direct access (or relative access). A file is made up of fixed-length logical records that allow programs to read and write records rapidly in no particular order. The direct-access method is based on a disk model of a file, since disks allow random access to any file block. For direct access, the file is viewed as a numbered sequence of blocks or records. A direct-access file allows arbitrary blocks to be read or written.

read n

write n

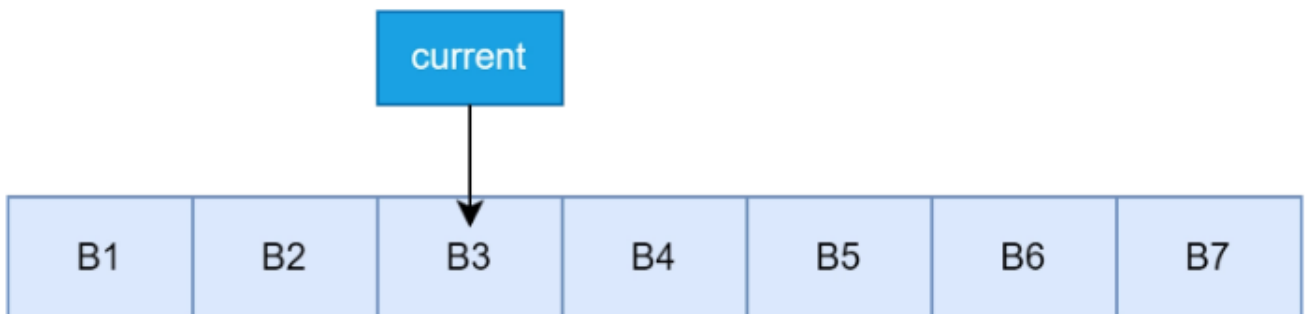
position to n

read next

write next

rewrite n

$n = \text{relative block number}$



File Attributes:

- **Name.** The symbolic file name is the only information kept in human readable form.
- **Identifier.** This unique tag, usually a number, identifies the file within the file system; it is the non-human-readable name for the file.
- **Type** This information is needed for systems that support different types of files.
- **Location.** This information is a pointer to a device and to the location of the file on that device.
- **Size** The current size of the file (in bytes, words, or blocks) and possibly the maximum allowed size are included in this attribute.
- **Protection.** Access-control information determines who can do reading, writing, executing, and so on.
- **Time, date, and user identification** This information may be kept for creation, last modification, and last use. These data can be useful for protection, security, and usage monitoring.

Information about files are kept in the directory structure, which is maintained on the disk.

File Operations:

- **Creating a file:** Two steps are necessary to create a file. First, space in the file system must be found for the file. Second, an entry for the new file must be made in the directory.
- **Writing a file:** To write a file, we make a system call specifying both the name of the file and the information to be written to the file.
- **Reading a file:** To read from a file, we use a system call that specifies the name of the file and where (in memory) the next block of the file should be put.
- **Repositioning within a file:** The directory is searched for the appropriate entry, and the current-file-position is set to a given value. Repositioning within a file does not need to involve any actual I/O. This file operation is also known as a file *seek*.
- **Deleting a file:** To delete a file, we search the directory for the named file. Having found the associated directory entry, we **release all file space**, so that it can be reused by other files, and erase the directory entry.

- **Truncating a file:** The user may **want to erase the contents of a file but keep its attributes**. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged-except for file length-but lets the file be reset to length zero and its file space released.

File System Layout:

The file system layout in an operating system (OS) refers to **how files and directories are organized and structured within the storage medium**, such as a hard disk drive or solid-state drive.

Different operating systems may have varying file system layouts, but there are some common concepts and components across many OS:

1. **Root Directory:** At the top level of the file system hierarchy is the root directory. It serves as the starting point for navigating the file system and contains other directories and files.
2. **Directories (Folders):** Directories are containers for organizing files and other directories. They can be nested within each other to create a hierarchical structure.

3. **Files:** Files are units of data stored on a storage medium. They can contain text, programs, images, documents, or any other type of data. Files are organized within directories.
4. **File Paths:** File paths are used to specify the location of a file within the file system. They consist of directory names separated by slashes ("/" in Unix-like systems, "\" in Windows) leading to the file.
5. **File System Metadata:** Each file and directory in the file system is associated with metadata such as permissions, creation date, last modified date, file size, and ownership information. This metadata is used by the operating system to manage and access files.

File Directories:

The collection of files is a file directory. The directory contains information about the files, including attributes, location, and ownership.

Much of this information, especially that is concerned with storage, is managed by the operating system.

Below are information contained in a device directory.

- Name
- Type
- Address
- Current length
- Maximum length
- Date last accessed
- Date last updated
- Owner id
- Protection information

Directory Operation:

- Search for a file
- Create a file
- Delete a file
- List a directory
- Rename a file
- Traverse the file system

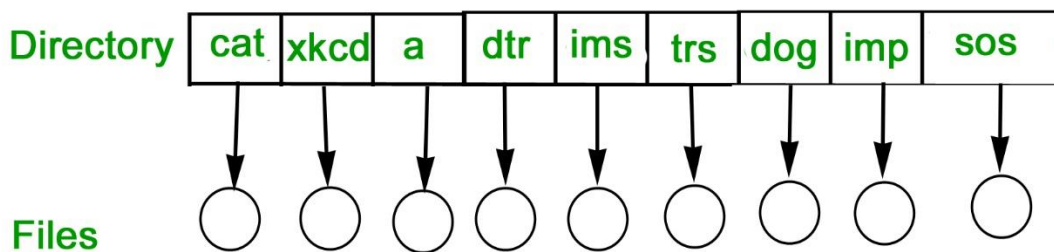
- **Create a file:** New files need to be created and added to the directory.
- **Delete a file:** When a file is no longer needed, we want to remove it from the directory.
- **List a directory:** We need to be able to list the files in a directory, and the contents of the directory entry for each file in the list.
- **Rename a file:** Because the name of a file represents its contents to its users, the name must be changeable when the contents or use of the file changes. Renaming a file may also allow its position within the directory structure to be changed.
- **Traverse the file system:** We may wish to access every directory, and every file within a directory structure. For reliability, it is a good idea to save the contents and structure of the entire file system at regular intervals

Most common schemes for defining the logical structure of a directory are:

Single-Level Directory:

In this, a single directory is maintained for all the users.

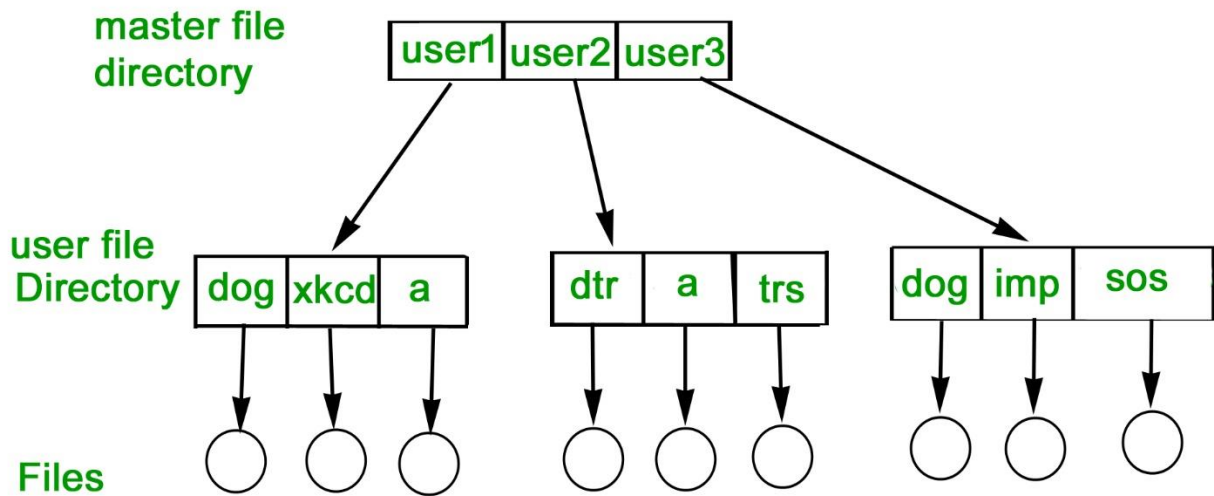
- **Naming problem:** Users cannot have the same name for two files.
- **Grouping problem:** Users cannot group files according to their needs.



Two-Level Directory:

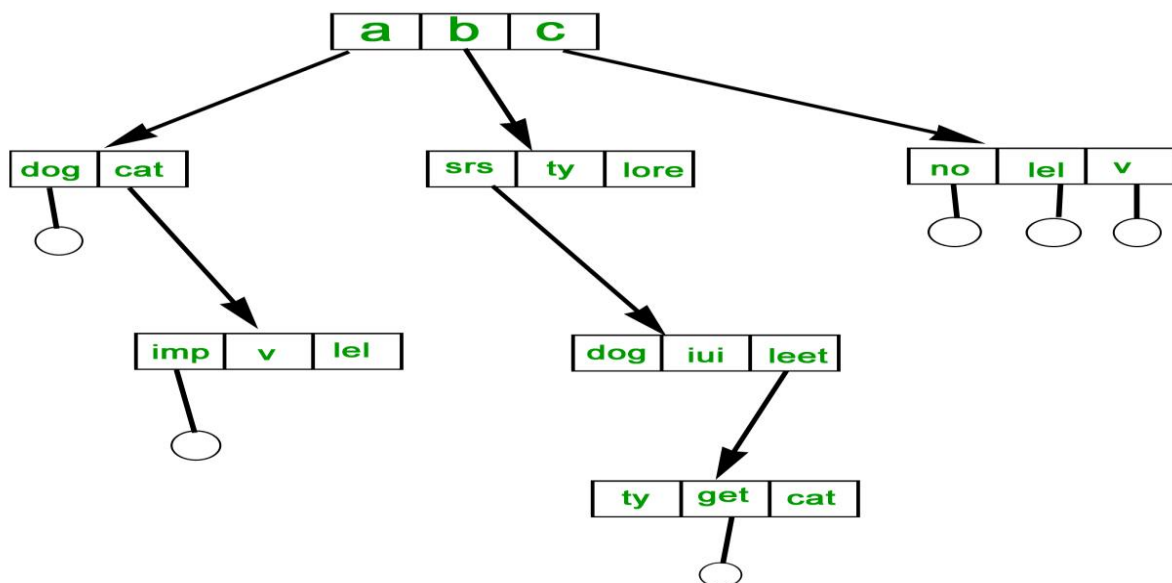
In this separate directories for each user is maintained.

- **Path name:** Due to two levels there is a path name for every file to locate that file.
- Now, we can have the same file name for different users.
- Searching is efficient in this method.



Tree-Structured Directory:

The directory is maintained in the form of a tree. Searching is efficient and also there is grouping capability. We have absolute or relative path name for a file.



Path Names:

When the file system is organized as a directory tree, some way is needed for specifying file names. Two different methods are commonly used:

1. **Absolute Path name** – In this method, each file is given an **absolute path** name consisting of the path from the root directory to the file.

As an **example**, the path **/usr/ast/mailbox** means that the root directory contains a subdirectory **usr**, which in turn contains a subdirectory **ast**, which contains the file **mailbox**.

Absolute path names always start at the root directory and are unique. In UNIX the components of the path are separated by **'/'**. In Windows, the separator is **'\'**. **Windows** **\usr\ast\mailbox** **UNIX** **/usr/ast/mailbox**

2. **Relative Path name** – A relative path is a path to a file or directory that is relative to the current working directory.

It does not start from the root directory but instead describes the location of the file or directory relative to the current location.

Directory Implementation:

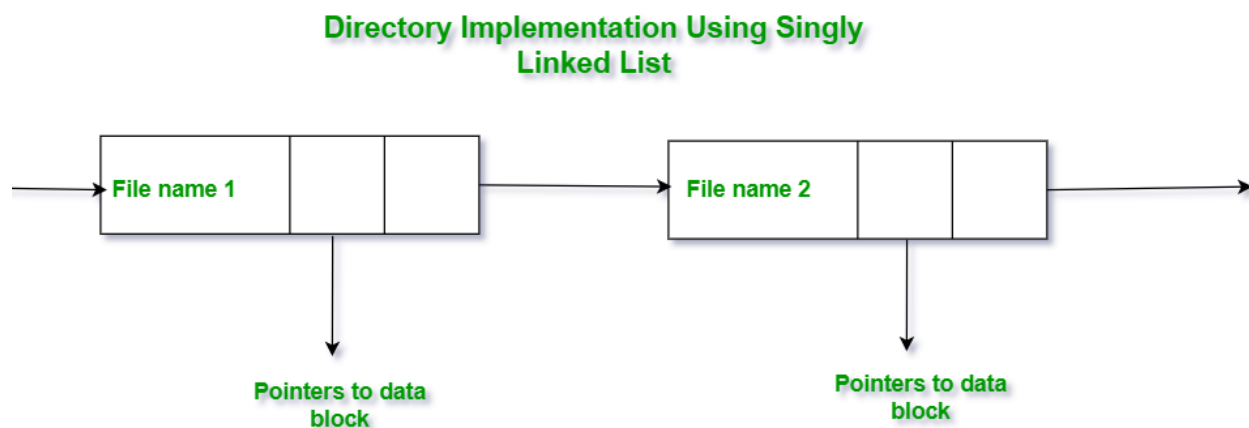
Directory implementation in the operating system can be done using Singly Linked List and Hash table.

The efficiency, reliability, and performance of a file system are greatly affected by the selection of directory-allocation and directory-management algorithms.

There are numerous ways in which the directories can be implemented. But we need to choose an appropriate directory implementation algorithm that enhances the performance of the system.

Directory Implementation using Singly Linked List:

The implementation of directories using a **singly linked list** is easy to program but is time-consuming to execute. Here we implement a directory by using a linear list of filenames with pointers to the data blocks.



Directory Implementation Using Singly Linked List

- To create a new file the entire list has to be checked such that the new directory does not exist previously.
- The new directory then can be added to the end of the list or at the beginning of the list.
- In order to delete a file, we first search the directory with the name of the file to be deleted. After searching we can delete that file by releasing the space allocated to it.
- To reuse the directory entry we can mark that entry as unused or we can append it to the list of free directories.

Disadvantage

The main disadvantage of using a linked list is that when the user needs to find a file the user has to do a linear search.

Directory Implementation using Hash Table:

An alternative data structure that can be used for directory implementation is a **hash table**.

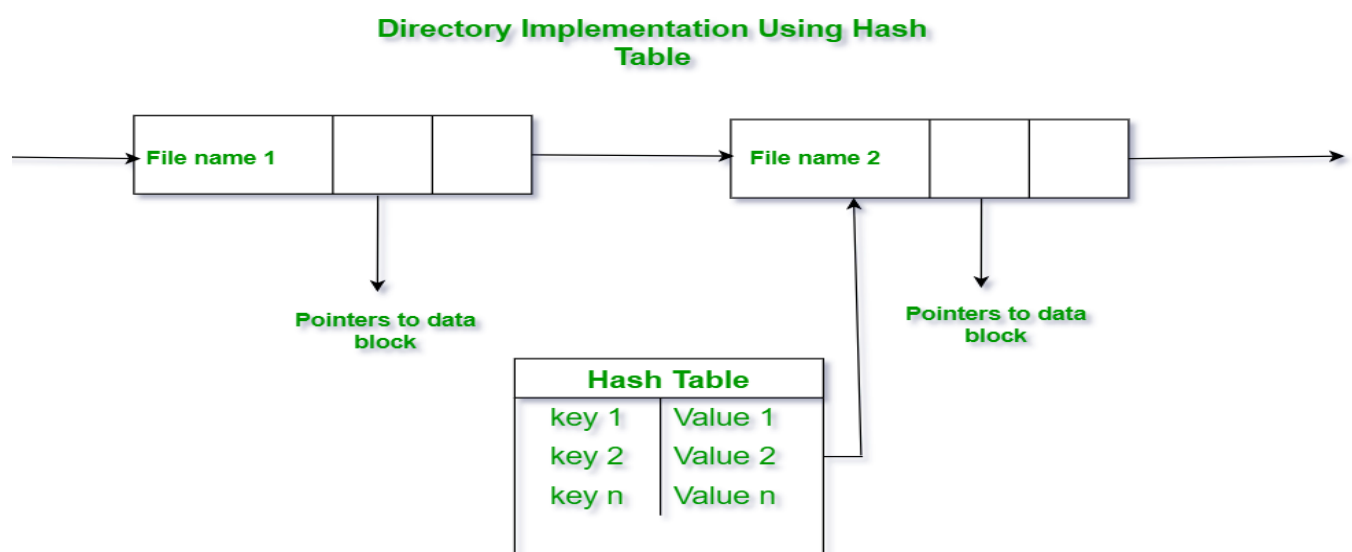
It **overcomes the major drawbacks** of directory implementation using a linked list.

In this method, we use a hash table along with the linked list. Here the linked list stores the directory entries, but a hash data structure is used in combination with the linked list.

In the **hash table for each pair in the directory key-value pair is generated.**

The hash function on the file name determines the key and this key points to the corresponding file stored in the directory.

This **method efficiently decreases the directory search time as the entire list will not be searched on every operation.** Using the keys the hash table entries are checked and when the file is found it is fetched.



Shared File:

A shared file refers to a file that is accessible and can be **modified or viewed by multiple users** or entities simultaneously.

In computing, shared files are commonly used in networked environments where multiple users need access to the same file or set of files.

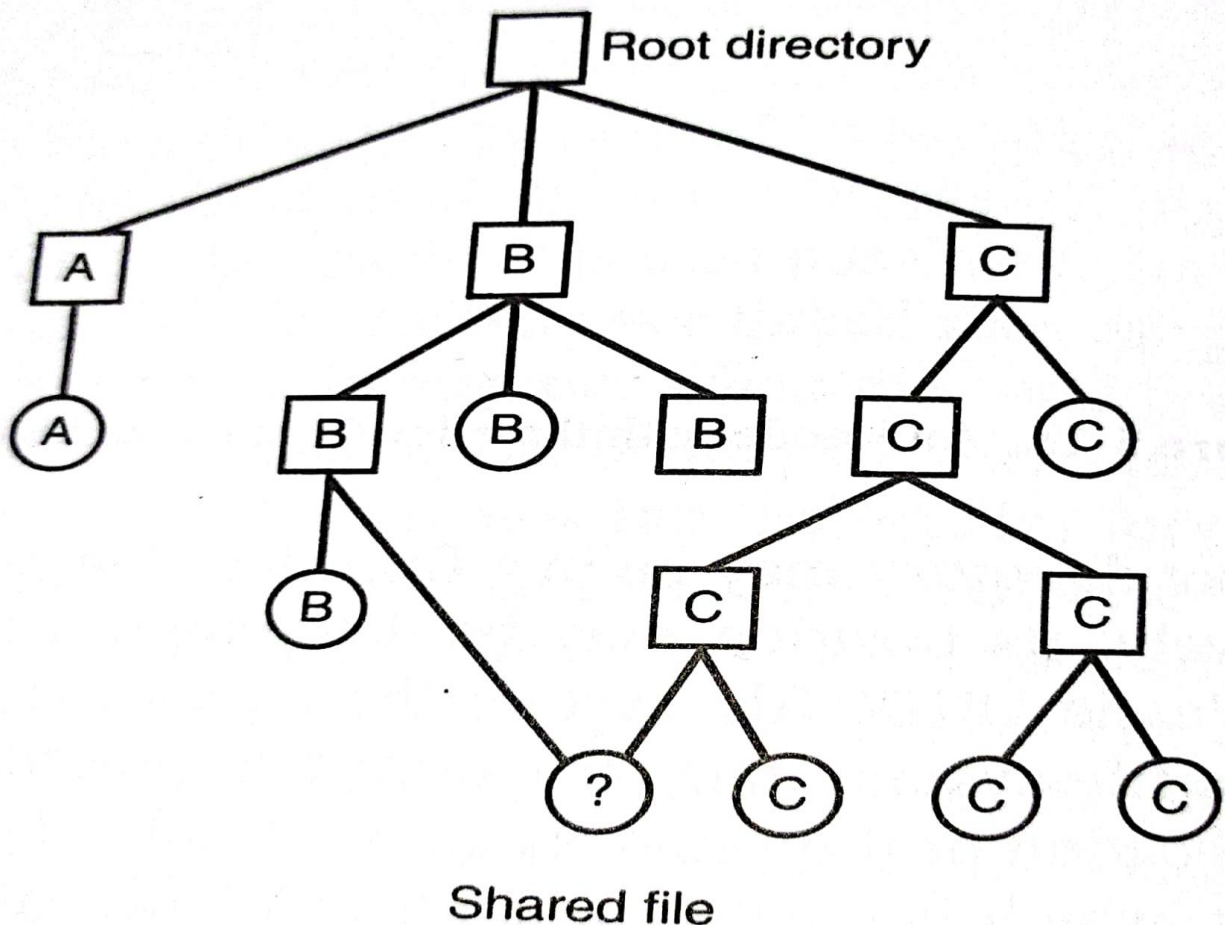


Figure: File system containing a shared file

Implementing Files:

Probably the most important issue in implementing file storage is keeping track of which disk blocks go with which file.

Various methods are used in different operating system are:

Contiguous Allocation:

- Each file occupies a contiguous address space on disk.
- Assigned disk address is in linear order.
- Easy to implement.
- External fragmentation is a major issue with this type of allocation technique (making it difficult to find contiguous blocks of space of sufficient length).

Linked List Allocation:

This method for storing files is to keep each one as a linked list of disk blocks.

The first word of each block is used as a pointer to the next one.

The rest of the block is for data.

- No external fragmentation
- Effectively used in sequential access file.
- Inefficient in case of direct access file.

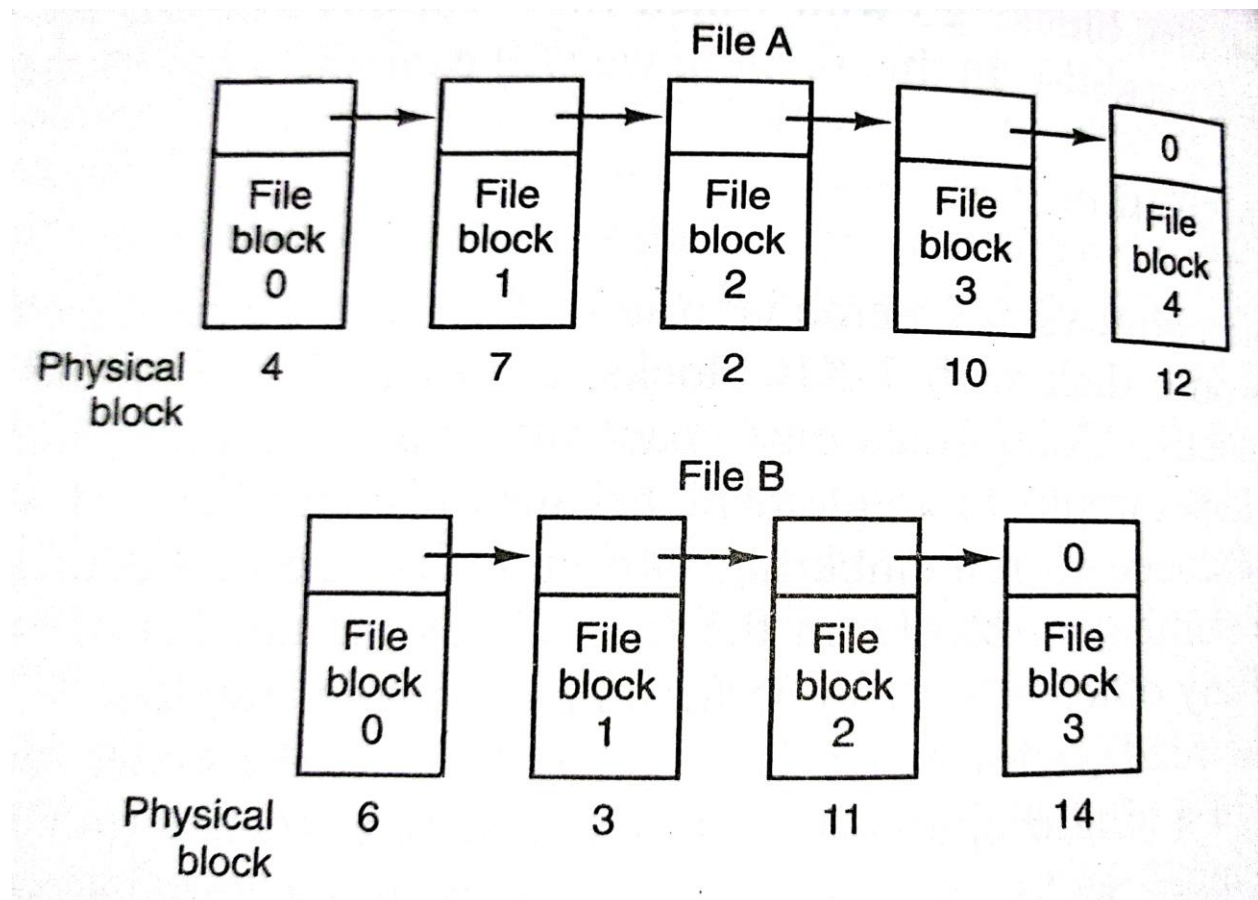


Figure: Storing a file as a linked list of disk blocks

Linked List Allocation Using a Table in Memory:

Linked-list allocation using a table in memory, often referred to as File Allocation Table (FAT), is a method used in file systems to manage disk space allocation efficiently.

This approach addresses some of the limitations of traditional linked-list allocation methods while introducing its own set of advantages and disadvantages.

how linked-list allocation using a FAT works:

1. **Basic Concept:** In traditional linked-list allocation, each file on disk is represented by a linked list of disk blocks. Each disk block contains a pointer to the next block in the file. However, traversing this linked list can be inefficient for random access operations since it requires sequentially reading through the blocks.
2. **File Allocation Table (FAT):** In the FAT approach, instead of storing pointers to disk blocks within the blocks themselves, a separate table known as the File Allocation Table is maintained in memory. This table maps each file's blocks to its logical sequence and stores the pointers to these blocks.

3. Structure of FAT: The FAT is typically implemented as an array, where each entry corresponds to a disk block. The value stored in each entry indicates the index of the next block in the file. For example, if entry 3 in the FAT contains the value 5, it means that the next block in the file is located at index 5 on the disk.

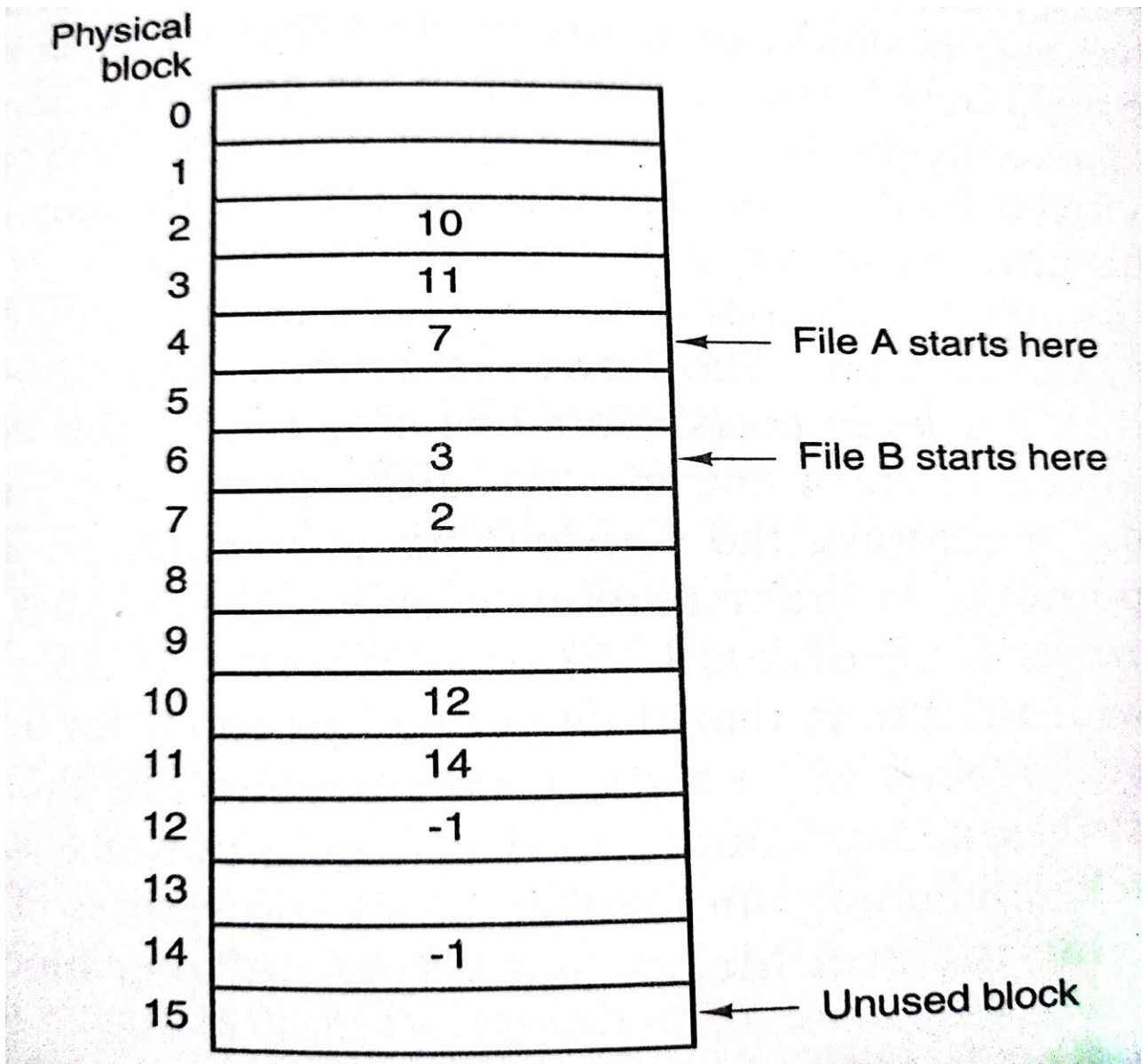
4. Advantages:

- **Random Access:** With FAT, random access to any block of a file becomes more efficient. Instead of traversing through the entire linked list, the file system can directly index into the FAT to find the location of the desired block.
- **Simplicity:** Managing disk space becomes simpler since the file system doesn't need to maintain pointers within each disk block. This can lead to faster file system operations.

5. Disadvantages:

- **Memory Overhead:** The primary disadvantage of using FAT is the memory overhead. The entire FAT table needs to be loaded into memory to enable efficient file access. For systems with large disks or limited memory, this can be impractical.

- **Single Point of Failure:** Since the FAT table resides entirely in memory, any corruption or loss of the FAT can lead to significant data loss or file system corruption. This makes the file system more susceptible to failures compared to distributed metadata approaches.



I-Node:

In UNIX based operating systems, each file is indexed by an Inode (index node).

Inode are the special disk block which is created with the creation of the file system.

The number of files or directories in a file system depends on the number of Inodes in the file system.

An Inode includes the following information

1. Attributes (permissions, time stamp, ownership details, etc) of the file
2. A number of direct blocks which contains the pointers to first 12 blocks of the file.
3. A single indirect pointer which points to an index block. If the file cannot be indexed entirely by the direct blocks then the single indirect pointer is used.
4. A double indirect pointer which points to a disk block that is a collection of the pointers to the disk blocks which are index blocks. Double index pointer is used if the file is too big to be indexed entirely by the direct blocks as well as the single indirect pointer.
5. A triple index pointer that points to a disk block that is a collection of pointers. Each of the pointers is separately pointing to a disk block which also contains

a collection of pointers which are separately pointing to an index block that contains the pointers to the file blocks.

